

Dynamic Role Assignment for Three-Agent Coordination in Overcooked-AI

Xingzhi Qian (xq87) Junzhe Huang (jh3229)
M.Eng. in Electrical and Computer Engineering
Cornell University
xq87@cornell.edu jh3229@cornell.edu

1 Introduction

Multi-agent coordination is a central problem in reinforcement learning and decision-making systems. In cooperative environments, agents must complete local actions while coordinating with teammates to avoid duplicated work, blocking, and inefficient task allocation. Overcooked-AI provides a useful testbed for this problem, where multiple agents jointly prepare and deliver soups in a shared kitchen.

This project studies three-agent coordination in a custom Overcooked-AI environment. Compared with the standard two-agent setting, adding a third agent introduces new coordination challenges: the extra agent may duplicate existing work, block critical locations, or interfere with the cooking-serving cycle. The main hypothesis of this project is that explicit dynamic role assignment can produce stable and interpretable three-agent coordination. To test this hypothesis, I design a hierarchical rule-based controller that assigns agents to *cook*, *serve*, and *support* roles using utility-based role assignment, role-specific policies, BFS-based navigation, and a joint-action conflict resolver.

In the nominal deterministic setting, the proposed controller consistently completes four soups within a 200-step horizon. I further evaluate the controller using a support-role delay stress test, which shows that the method is effective in the nominal setting but remains sensitive to longer early-stage support-role disruptions.

2 Problem Formulation

2.1 Cooperative Markov Game Formulation

The Overcooked-AI environment can be formulated as a cooperative Markov game, which is a multi-agent extension of a Markov Decision Process (MDP). In a standard MDP, an agent observes a state, chooses an action, receives a reward, and transitions to the next state. In this project, three agents act simultaneously, so the single-agent action is replaced by a joint action.

Formally, the three-agent environment is represented by:

$$\mathcal{M} = (\mathcal{S}, \mathcal{A}_0, \mathcal{A}_1, \mathcal{A}_2, P, R, H),$$

where $\mathcal{S}$ is the state space, $\mathcal{A}_i$ is the action space of agent i , P is the transition function, R is the shared team reward function, and H is the finite episode horizon.

At timestep t , the environment is in state $s_t \in \mathcal{S}$. Each agent i selects an action $a_t^i \in \mathcal{A}_i$, forming the joint action:

$$\mathbf{a}_t = (a_t^0, a_t^1, a_t^2).$$

The environment then transitions according to:

$$s_{t+1} \sim P(s_{t+1} \mid s_t, \mathbf{a}_t),$$

and the team receives a shared reward:

$$r_t = R(s_t, \mathbf{a}_t, s_{t+1}).$$

Because all agents share the same team objective, the goal is cooperative rather than competitive: agents must coordinate their joint behavior to maximize the total team return.

2.2 State, Action, and Reward

The state s_t contains the current kitchen configuration, including agent positions and orientations, held objects, pot status, fixed object locations, walls, and walkable cells. This satisfies the Markov property because the next state depends only on the current state and current joint action:

$$P(s_{t+1} \mid s_0, \mathbf{a}_0, \dots, s_t, \mathbf{a}_t) = P(s_{t+1} \mid s_t, \mathbf{a}_t).$$

For example, whether an agent should fetch an onion, prepare a dish, or wait can be determined from the current pot state, held objects, and agent positions.

Each agent has the same discrete action space:

$$\mathcal{A}_i = \{\text{North, South, East, West, Stay, Interact}\}.$$

The movement actions change the agent’s position if the target cell is valid. The `Stay` action keeps the agent in place, and `Interact` is used to pick up objects, place onions into the pot, start cooking, collect soup, or deliver soup depending on the agent’s location and orientation.

Since there are three agents, the joint action space is:

$$\mathcal{A} = \mathcal{A}_0 \times \mathcal{A}_1 \times \mathcal{A}_2.$$

With six actions per agent, the joint action space has:

$$|\mathcal{A}| = 6^3 = 216$$

possible joint actions at each timestep, which illustrates why coordination becomes more difficult as the number of agents increases.

The environment provides a shared team reward. The sparse reward is given when a soup is successfully delivered:

$$r_t^{\text{sparse}} = \begin{cases} 20, & \text{if a soup is delivered at timestep } t, \\ 0, & \text{otherwise.} \end{cases}$$

The environment also provides shaped rewards for useful intermediate progress, such as ingredient handling and dish preparation. The total return over an episode is:

$$G = \sum_{t=0}^{H-1} r_t.$$

In the main experiment, the horizon is $H = 200$, and the practical objective is to maximize the number of soups delivered within this horizon.

2.3 Policy Objective and Coordination Challenge

A joint policy maps the current state to a joint action:

$$\pi(s_t) = \mathbf{a}_t.$$

In a learning-based reinforcement learning setting, the goal would be to learn a policy that maximizes expected return:

$$\pi^* = \arg \max_{\pi} \mathbb{E}_{\pi} \left[\sum_{t=0}^{H-1} r_t \right].$$

In this project, instead of training an end-to-end neural policy, I design and evaluate an interpretable hierarchical joint policy:

$$\pi(s_t) = \pi_{\text{low}}(s_t, \pi_{\text{role}}(s_t)),$$

where π_{role} is the high-level role assignment policy and π_{low} is the low-level role-specific action policy.

The three-agent setting is non-trivial because adding an extra agent can create role ambiguity, duplicated work, bottleneck blocking, and joint-action conflicts. Therefore, the main design idea is to reduce coordination ambiguity by explicitly assigning agents to three roles: *cook*, *serve*, and *support*.

3 Method

3.1 Custom Three-Agent Layout

This project uses a custom three-agent Overcooked-AI layout containing three agents, an onion source, a pot, a dish source, and a serving location. The task is to repeatedly fetch onions, fill the pot, prepare dishes, collect ready soup, and deliver it within a fixed episode horizon.

Figure 1 shows the symbolic layout printed from the environment initialization cell. This representation verifies the exact grid structure, static object locations, and initial agent positions.

```
Environment initialized.
Layout: custom_role_split_3agents
Horizon: 200
Number of players detected: 3

=== Custom 3-Agent Layout ===
Legend: X=wall, O=onion, P=pot, D=dish, S=serving
Agents: ↑/↓/←/→ show orientation, number shows agent index

  X  X  X  X  X  X  X  X  X  X  X
  X  ↑0      O      P      D      X
  X                    ↑2      X
  X                    ↑1      S      X
  X  X  X  X  X  X  X  X  X  X  X
```

Figure 1: Printed symbolic representation of the custom three-agent layout. The map verifies the grid structure, static object locations, and initial agent positions.

Figure 2 shows the same environment rendered with the interactive grid-world visualization. This view is used to inspect agent movement, assigned roles, held objects, pot status, and reward progress during an episode.

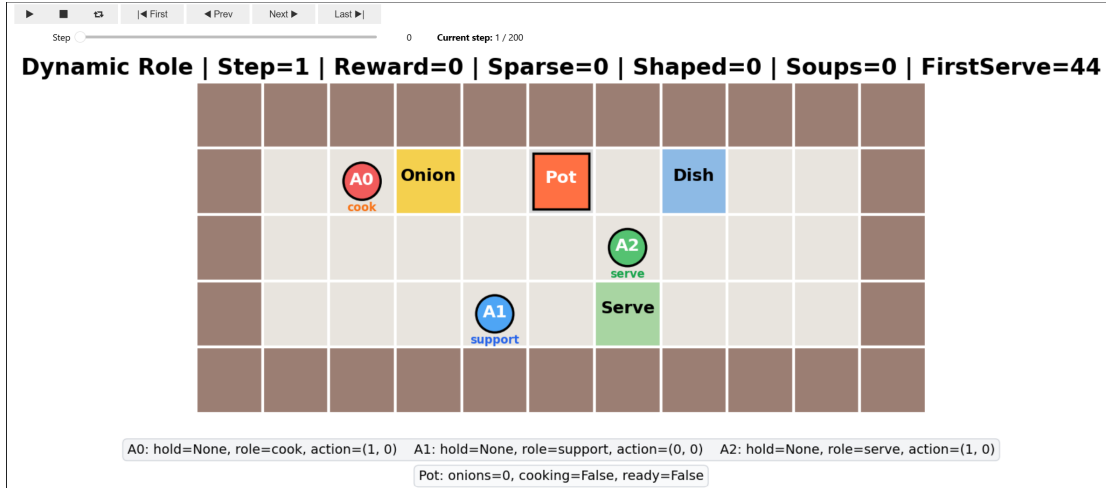


Figure 2: Interactive grid-world visualization of the custom three-agent Overcooked-AI environment. The visualization shows agents, roles, objects, pot status, and episode information.

This custom layout creates a simple but meaningful coordination problem. A third agent can potentially improve efficiency, but it can also cause duplicated work or block critical cells. Therefore, the controller must explicitly coordinate the agents rather than allowing each agent to act independently.

3.2 Hierarchical Controller

The proposed method is a hierarchical rule-based controller. Instead of directly selecting a low-level action for each agent independently, the controller first assigns each agent to a high-level role and then generates role-specific actions. The controller has four main components:

1. **Utility-based dynamic role assignment:** each agent receives utility scores for the *cook*, *serve*, and *support* roles, and the controller selects the assignment that maximizes total team utility.
2. **Role-specific policies:** after roles are assigned, each agent follows a rule-based policy corresponding to its role.
3. **BFS-based navigation:** each agent uses shortest-path navigation to move toward its target location.
4. **Joint-action conflict resolver:** before execution, the controller checks and corrects simple same-cell and swap conflicts.

The overall decision pipeline is shown in Figure 3. At each timestep, the controller observes the current Markov state, computes role utilities, assigns roles, generates role-specific actions, resolves conflicts, and outputs a three-agent joint action to the environment.

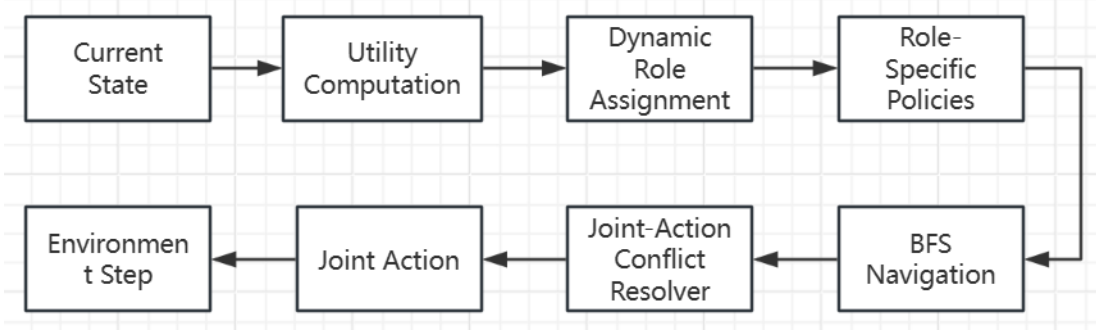


Figure 3: Overview of the hierarchical three-agent controller. The controller observes the current state, computes role utilities, assigns agents to cook, serve, and support roles, generates role-specific actions with BFS navigation, resolves joint-action conflicts, and outputs a three-agent joint action.

This structure separates high-level coordination from low-level movement. The role assignment determines what each agent should focus on, while the low-level policies determine how each assigned sub-task is executed.

3.3 Dynamic Role Assignment and Low-Level Execution

At each timestep, the controller computes a utility score for assigning each agent to each role:

$$U_i(\text{cook}), \quad U_i(\text{serve}), \quad U_i(\text{support}).$$

It then enumerates all valid permutations of the three roles and selects the assignment that maximizes total utility:

$$\rho^*(s_t) = \arg \max_{\rho \in \mathcal{R}} \sum_{i=0}^2 U_i(\rho_i; s_t),$$

where ρ_i is the role assigned to agent i and $\mathcal{R}$ is the set of all permutations of $\{\text{cook}, \text{serve}, \text{support}\}$.

The utility functions use task-relevant features of the current state, including pot status, held objects, distances to important locations, and bottleneck penalties. The cook utility increases when the pot needs onions or when the agent is holding an onion. The serve utility increases when the pot is ready or when the agent is holding a dish or soup. The support utility increases when the pot is close to completion and a dish should be prepared in advance.

After roles are assigned, each agent executes a role-specific policy. The **cook** agent fetches onions, places them into the pot, and starts cooking when the pot is full. The **serve** agent prepares dishes, collects ready soup, and delivers it to the serving location. The **support** agent acts as an auxiliary coordinator by prefetching dishes or moving to buffer and safe locations when no immediate task is needed.

For low-level movement, the controller uses breadth-first search (BFS) to compute the next step toward the current target, such as the onion source, pot, dish source, serving location, or waiting spot. The planner treats other agents' current positions as blocked cells to reduce direct collisions.

Because all agents act simultaneously, individual BFS planning alone cannot prevent all joint-action conflicts. Therefore, the controller applies a conflict resolver after individual actions are generated. It handles two common cases: **same-target conflicts**, where multiple agents intend to move into the same cell, and **swap conflicts**, where two agents intend to exchange positions. When a conflict is detected, the lower-priority agent is forced to stay. If the pot is ready, the serve role receives highest priority; if the pot still needs onions, the cook role receives highest priority; the support role usually has the lowest priority.

4 Experimental Setup

The proposed controller is evaluated in the custom three-agent Overcooked-AI environment with a finite horizon of 200 timesteps. At each timestep, the controller observes the current Markov state, assigns roles, generates a joint action, and receives sparse and shaped rewards from the environment. The evaluation contains two parts: a nominal deterministic rollout and a support-role delay stress test.

Table 1: Experimental setup for the three-agent coordination task.

Item	Setting
Environment	Overcooked-AI
Task setting	Cooperative soup preparation and delivery
Number of agents	3
Layout	Custom three-agent layout
Episode horizon	200 timesteps
Policy type	Utility-based hierarchical rule-based controller
High-level decision	Dynamic role assignment
Low-level navigation	BFS shortest-path planning
Conflict handling	Same-cell and swap-conflict resolver
Main evaluation	Nominal deterministic rollout
Additional evaluation	Support-role delay stress test

The main task-level metrics are sparse reward, shaped reward, number of soups served, and first serve step. I also use diagnostic visualizations, including role assignment timelines, pot status timelines, and reward curves, to analyze whether the controller forms a stable division of labor and repeated cooking-serving cycle.

In the nominal evaluation, the controller is executed for 10 repeated episodes under the same initial condition and deterministic policy. Since both the environment and the controller are deterministic, repeated episodes produce identical trajectories. Therefore, the multi-episode evaluation is used to verify reproducibility under the nominal setting rather than stochastic robustness.

To examine the controller beyond the nominal setting, I introduce a support-role delay stress test. During the first few timesteps of an episode, whichever agent is currently assigned to the support role is forced to take the `Stay` action. The delay length is varied over:

$$0, 5, 10, 15, 20$$

timesteps. This test evaluates whether the controller can recover from early disruption of the support role, which is responsible for dish prefetching and bottleneck reduction.

5 Results

5.1 Overall Performance

Table 2 summarizes the nominal performance of the proposed three-agent dynamic role controller. The controller is evaluated over 10 repeated episodes with the same initial condition and deterministic policy. Since

both the environment and the controller are deterministic, the standard deviation is zero across repeated runs. Therefore, the repeated evaluation mainly verifies reproducibility under the nominal setting.

The controller consistently completes four soups within the 200-step horizon. Since each soup delivery gives a sparse reward of 20, four successful deliveries correspond to a total sparse reward of 80. The first soup is delivered at step 44, indicating that the controller quickly establishes the first cooking-serving cycle.

Table 2: Nominal performance of the three-agent dynamic role controller over 10 repeated episodes.

Metric	Value
Average sparse reward	80.00
Standard deviation of sparse reward	0.00
Average shaped reward	65.00
Standard deviation of shaped reward	0.00
Average episode length	200.00
Average soups served	4.00
Average first serve step	44.00

These results show that the proposed controller can produce stable and repeatable three-agent coordination in the custom Overcooked-AI layout.

To better contextualize the nominal performance of the proposed controller, I also compare it with a simpler two-agent baseline. This comparison helps isolate the effect of introducing the third agent and the support role. In particular, the comparison addresses whether the observed performance gain comes merely from adding one more agent or from the explicit dynamic role structure itself.

Table 3: Comparison between the two-agent baseline and the proposed three-agent dynamic role controller.

Method	# Agents	Sparse reward	Shaped reward	Soups served
Two-agent baseline	2	40	51	2
Two-agent dynamic-role	2	60	71	3
Three-agent dynamic role controller	3	80	65	4

The comparison shows whether the support role provides a genuine coordination benefit beyond the standard two-agent division of labor. If the three-agent controller consistently achieves higher sparse reward or completes more soups, then the gain can be attributed not only to additional manpower but also to more effective task decomposition and coordination.

5.2 Role Assignment Behavior

Figure 4 shows the role assignment timeline during one representative episode. The timeline indicates that the controller forms a stable division of labor. Agent 0 consistently takes the *cook* role, while Agents 1 and 2 dynamically alternate between the *serve* and *support* roles.

This pattern supports the main hypothesis of the project: explicit role assignment can make three-agent coordination more structured and interpretable. Instead of all agents independently choosing low-level actions without coordination, the controller assigns each agent a clear responsibility at each timestep.

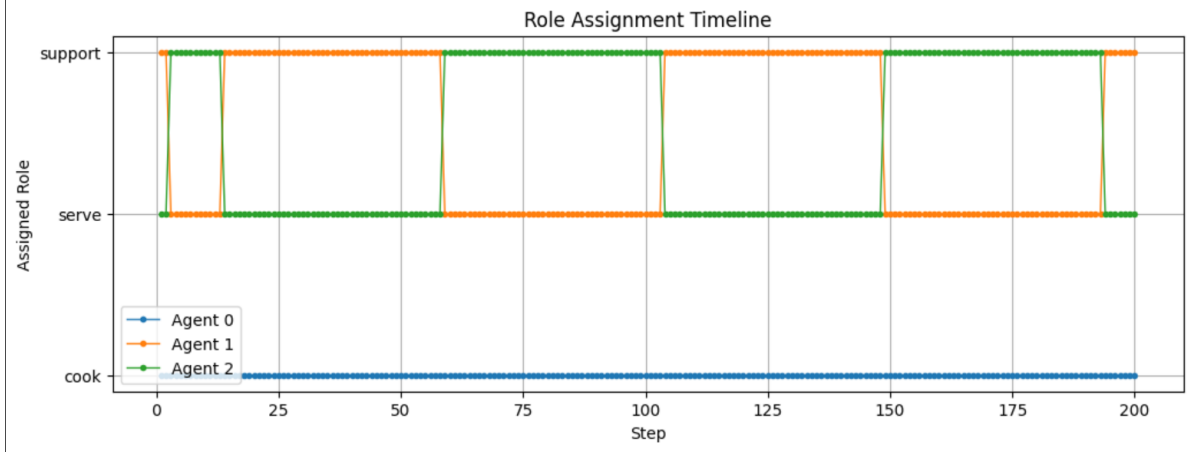


Figure 4: Role assignment timeline. Agent 0 consistently takes the cook role, while Agents 1 and 2 alternate between serve and support roles, forming a stable division of labor.

5.3 Pot Status and Cooking-Serving Cycle

Figure 5 shows the pot status over time. The blue curve represents the number of onions in the pot, and the orange curve indicates whether the pot is ready. The pot follows a repeated cycle: onions are added until the pot reaches three onions, the soup becomes ready, the soup is collected and delivered, and the pot state resets.

This repeated pattern shows that the controller does not merely complete the task once by chance. Instead, it maintains a stable production pipeline across the episode. The cook role keeps filling the pot, while the serve and support roles help prepare dishes and deliver soups when the pot becomes ready.

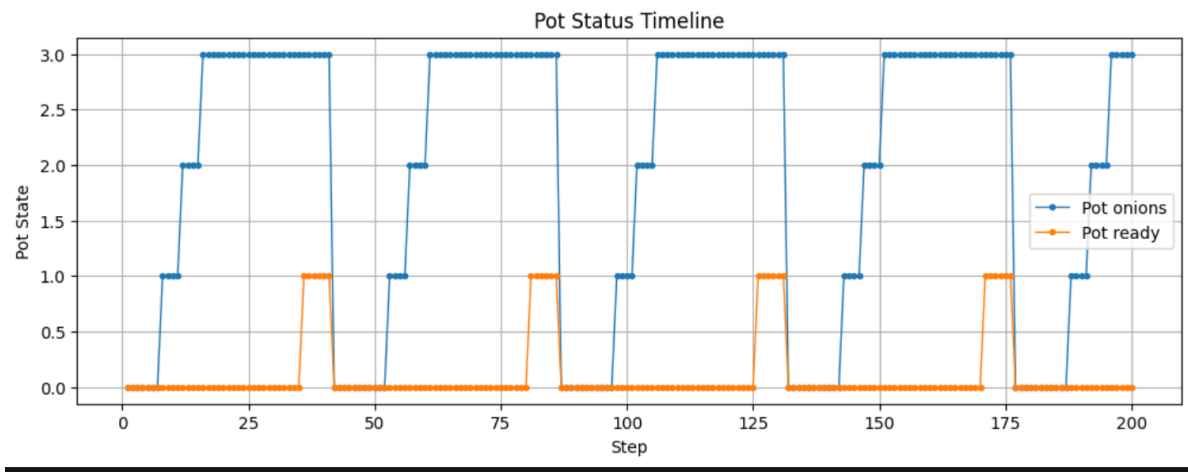


Figure 5: Pot status timeline. The pot repeatedly fills from 0 to 3 onions, becomes ready, and resets after soup delivery, showing a stable cooking-serving cycle.

5.4 Representative Episode Snapshots

To further illustrate the behavior of the controller, Figure 6 shows four representative snapshots from the first cooking-serving cycle of one episode. These snapshots correspond to early coordination, pot filling and

cooking, soup readiness, and the first successful delivery.

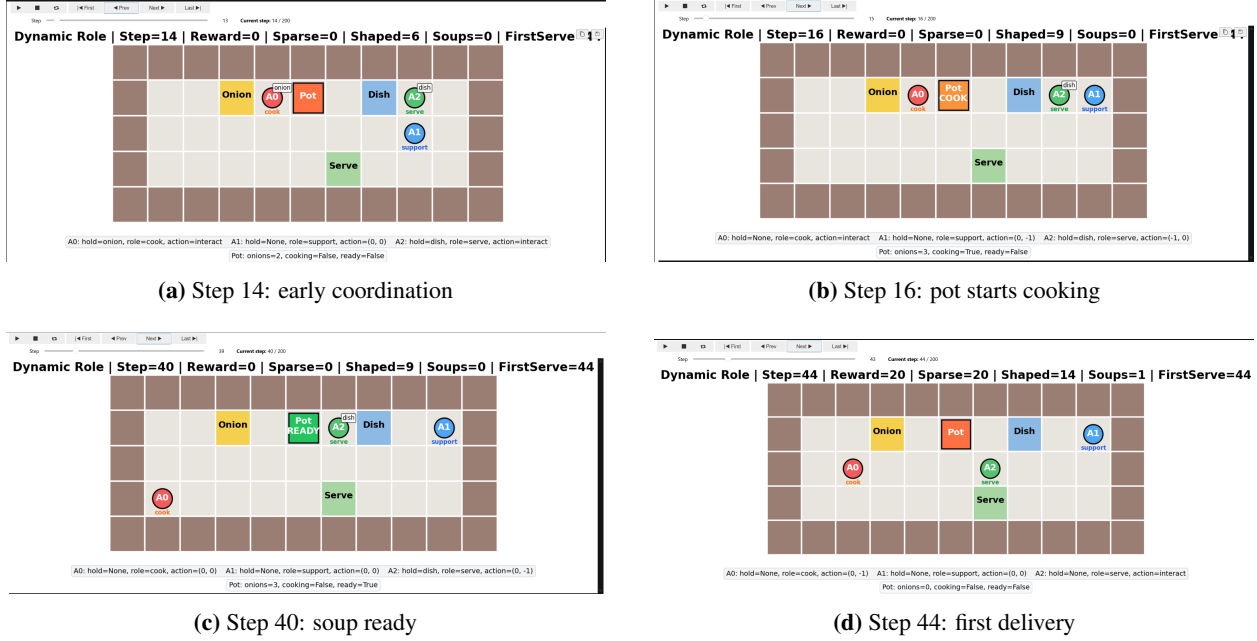


Figure 6: Representative snapshots from the first cooking-serving cycle of one episode. (a) At step 14, the controller has already established an initial division of labor: the cook agent is handling onions, the serve agent has prepared a dish, and the support agent remains near the serving side. (b) At step 16, the pot reaches three onions and starts cooking, indicating that the cooking pipeline has been formed. (c) At step 40, the soup becomes ready, and the serve agent with a dish is positioned near the pot. (d) At step 44, the first soup is successfully delivered, producing a sparse reward of 20.

The four snapshots in Figure 6 visualize one complete cooking-serving cycle, from initial coordination to the first successful soup delivery. In contrast, Figure 7 shows the final episode state after 200 timesteps, confirming that the controller maintains the coordination pattern throughout the full episode and completes four soup deliveries in total.

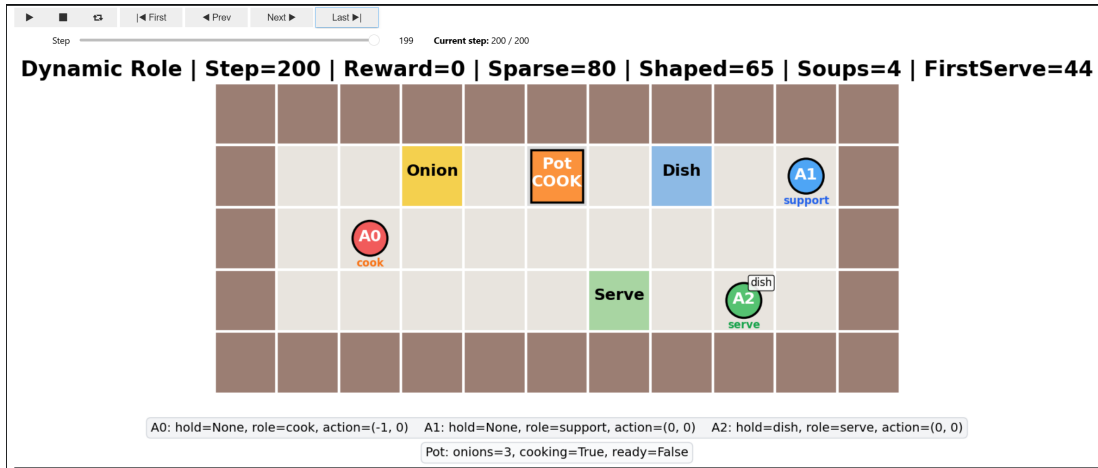


Figure 7: Final episode state at step 200. By the end of the episode, the controller has completed 4 soups, achieved a total sparse reward of 80, and accumulated a shaped reward of 65.

5.5 Reward Accumulation

Figure 8 shows the cumulative sparse and shaped rewards over the episode. The cumulative sparse reward increases in a step-wise pattern from 0 to 80. Each jump corresponds to one successful soup delivery. Since there are four jumps, the reward curve confirms that the controller completes four soups within the 200-step horizon.

The shaped reward increases more gradually because it is accumulated through intermediate sub-tasks, such as ingredient handling and dish preparation. Together, the sparse and shaped reward curves show that the agents not only complete final deliveries, but also make consistent progress during the preparation process.

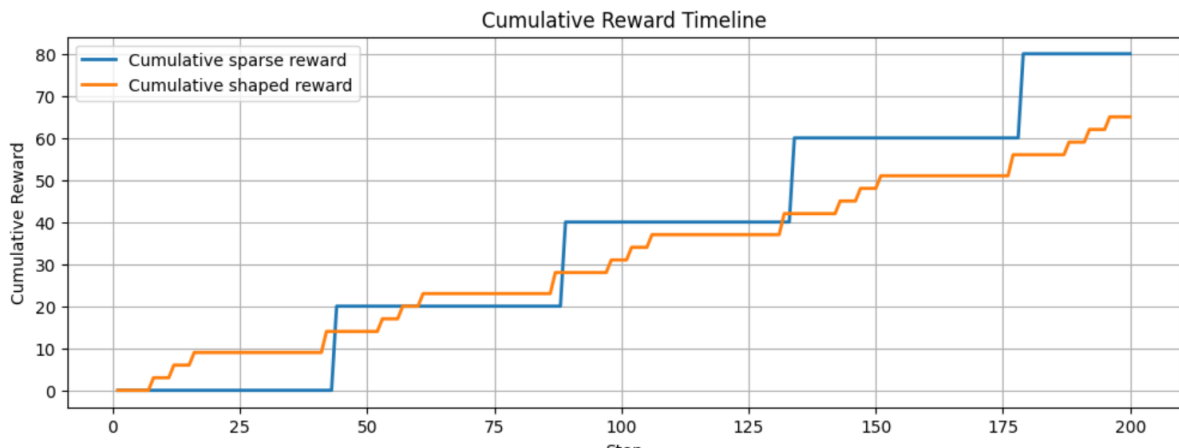


Figure 8: Cumulative reward timeline. The sparse reward increases stepwise with each soup delivery, while the shaped reward grows through intermediate task progress.

Overall, the results support the main hypothesis that explicit dynamic role assignment can produce stable and interpretable three-agent coordination in the custom Overcooked-AI environment. The controller forms a clear division of labor, repeatedly completes the cooking-serving cycle, and achieves four soup deliveries within the 200-step horizon.

6 Support-Role Delay Stress Test

The nominal results show that the proposed controller can produce stable and repeatable coordination under the default deterministic setting. However, this does not necessarily mean that the controller is robust to disruptions. To further examine the limitation of the controller, I introduce a support-role delay stress test.

In this test, the agent currently assigned to the *support* role is forced to take the *Stay* action during the first few timesteps of the episode. The delay length is varied over 0, 5, 10, 15, and 20 timesteps. The goal is to test whether the controller can recover when the early support behavior is disrupted.

Table 4: Support-role delay stress test results.

Initial support-role delay	Sparse reward	Soups served
0 steps	80	4
5 steps	80	4
10 steps	0	0
15 steps	0	0
20 steps	0	0

Figures 9 and Figures 10 show the effect of the initial support-role delay on sparse reward and the number of soups served. The controller maintains the same performance when the support role is delayed by 5 steps, still completing 4 soups and achieving a sparse reward of 80. However, when the support-role delay reaches 10 steps or more, the controller fails to complete any soup within the 200-step horizon.

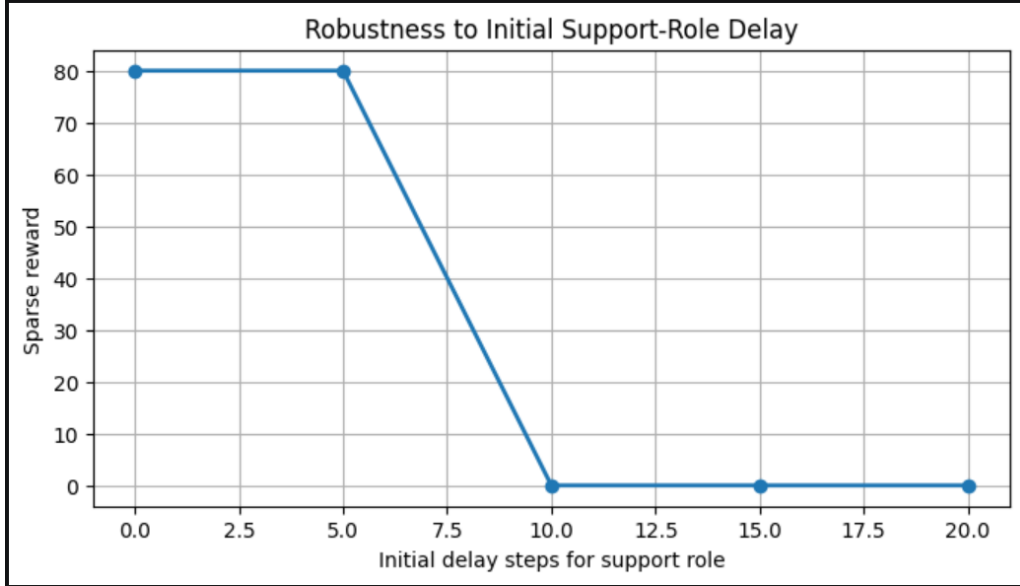


Figure 9: Stress test under initial support-role delay. The sparse reward remains 80 under a 5-step delay but drops to 0 when the delay reaches 10 steps or more.

A notable feature of Table 4 is that the degradation is not gradual. The controller retains full nominal performance under a 5-step delay, but completely fails when the delay reaches 10 steps or more. This suggests that the support role is not merely providing small incremental help; instead, it is critical for establishing the initial cooking-serving pipeline during the early part of the episode.

One possible explanation is that the support agent is responsible for early dish prefetching and bottleneck reduction at exactly the stage when the first soup cycle is being formed. If this early support behavior is delayed only slightly, the system can still recover and synchronize the cooking and serving roles. However, once the delay exceeds a certain threshold, the timing of dish preparation and movement coordination becomes misaligned, preventing the first successful cooking-serving cycle from forming within the 200-step horizon.

Therefore, the stress test indicates a threshold-type failure mode rather than a smooth performance decline. In other words, the current controller is robust to small early disruptions, but it is sensitive to larger coordi-

nation delays that prevent the initial role structure from stabilizing.

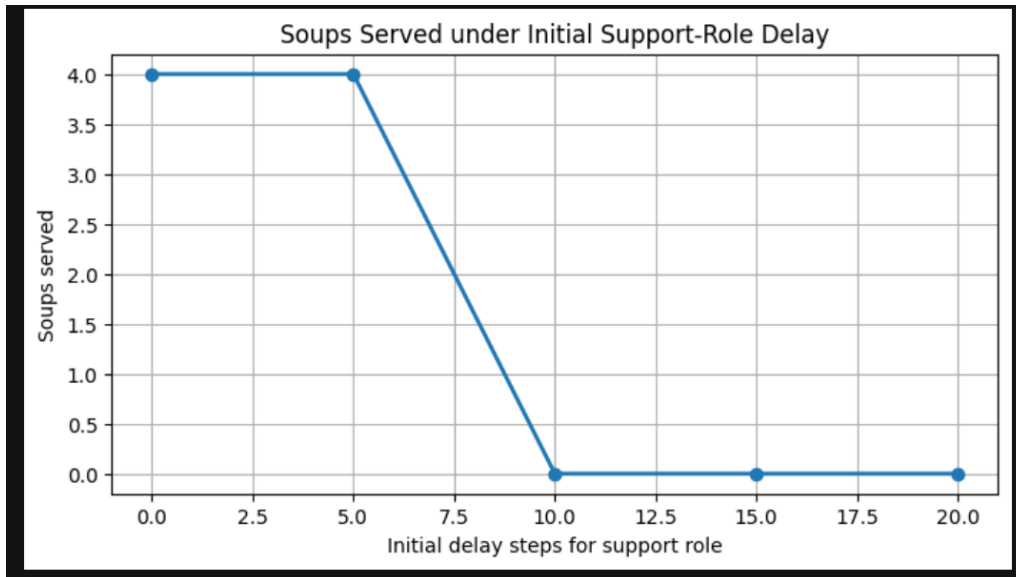


Figure 10: Soups served under initial support-role delay. The controller completes 4 soups under 0-step and 5-step delays, but completes no soup under longer delays.

These results reveal an important limitation of the proposed controller. The support role is important for establishing the early cooking-serving cycle, especially for preparing dishes and reducing bottleneck interference. A short disruption can be tolerated, but a longer disruption prevents the system from forming the normal coordination pattern. This suggests that the current rule-based controller is effective in the nominal setting but lacks a strong recovery mechanism under longer early-stage coordination failures.

This stress test therefore supports the secondary hypothesis: explicit role assignment can produce stable coordination under normal conditions, but the controller remains sensitive to disruptions that interfere with the early support behavior. Future work should include recovery logic, learned high-level role reassignment, or reinforcement learning fine-tuning to improve robustness.

7 Discussion and Future Work

This project shows that explicit role assignment can provide an interpretable structure for three-agent coordination in Overcooked-AI. The proposed controller achieves stable performance in the nominal setting and produces clear role, pot, and reward patterns. The result suggests that a structured policy can reduce role ambiguity and duplicated work in a three-agent cooperative task.

However, the support-role delay stress test reveals an important limitation. The controller does not have a strong recovery mechanism when early support timing is disrupted for too long. This limitation is expected because the policy is rule-based and does not learn from failed trajectories.

Future work can extend this project in several directions. First, the rule-based controller can be used as an expert to generate demonstrations for behavior cloning. Second, a learned high-level role classifier can replace the hand-designed utility functions. Third, reinforcement learning fine-tuning can be used to improve recovery behavior under disruptions. Finally, the controller can be evaluated on more layouts with tighter bottlenecks and different object placements.

Overall, the proposed controller serves as an interpretable expert policy and a foundation for future learning-based three-agent coordination in Overcooked-AI.

8 Group Contribution and AI Tool Usage

Xingzhi Qian contributed to the baseline controller design and the final visualization system. He first implemented and tested the two-agent baseline setting, designed the initial coordination logic, and developed the final visual analysis tools, including the interactive grid-world visualization, role assignment timeline, pot status timeline, reward curves, and representative episode snapshots.

Junzhe Huang contributed to the three-agent extension. Building on the baseline controller, he introduced the third agent as a support role, refined the three-agent coordination logic, and tested the controller until it successfully completed the cooperative cooking task in the custom three-agent environment.

Both members jointly worked on the final report, including the problem formulation, method description, experimental analysis, and discussion of limitations.

AI tools were used to assist with debugging, interactive visualization design, controller logic refinement, and language polishing for the report. All final implementation choices, experimental interpretations, and project-level decisions were reviewed and finalized by the team members.

References

1. Carroll, M., Shah, R., Ho, M. K., Griffiths, T., Seshia, S., Abbeel, P., and Dragan, A. “On the Utility of Learning about Humans for Human-AI Coordination.” *Advances in Neural Information Processing Systems*, 2019.
2. Overcooked-AI: https://github.com/HumanCompatibleAI/overcooked_ai
3. Sutton, R. S., and Barto, A. G. *Reinforcement Learning: An Introduction*. MIT Press, 2018.